

LLM-Assisted Big Data Integration

Traditional and LLM-Assisted Pipelines

Juan Miguel Pinos Seco

Advanced Topics in Computer Science – Roma Tre University

June 2026

Abstract

This report compares a traditional Big Data Integration pipeline with an LLM-assisted alternative for product data. Both pipelines cover schema alignment, record linkage, clustering, and data fusion. The traditional approach uses lexical similarity, blocking, weighted matching, Union-Find clustering, and majority voting. The second pipeline uses Qwen2.5-7B-Instruct to support schema alignment, borderline record-linkage decisions, and selected data-fusion cases.

The experiment contains 1,800 records, 300 entities, three heterogeneous source views, 3,600 positive cross-source pairs, and 1,220 conflicting entity-attribute cases. The LLM improves schema-alignment F1 from 0.571 to 1.000. In record linkage, precision increases slightly from 0.305 to 0.318, while recall decreases from 0.655 to 0.578. The final integrated-value accuracy increases from 0.200 to 0.239, although this measure also reflects differences in schema alignment, linkage, and clustering.

The results show that the LLM is especially useful for semantic schema interpretation. Its contribution is less consistent in record linkage and generative fusion, where deterministic validation and fallback rules remain necessary.

1 Introduction

Big Data Integration is the process of combining information from different sources into a common and consistent representation. This process is difficult because different sources may describe the same real-world entities using different attribute names, formats, languages, and values. For example, the attributes `manufacturer`, `brand_name`, and `maker` may all represent the same concept, even though their names are different.

A complete integration process usually includes three main stages. The first is *schema alignment*, where the attributes of each source are mapped to a shared mediated schema. The second is *record linkage*, also called entity resolution, where records referring to the same real-world entity are identified. The third is *data fusion*, where duplicated or conflicting values are combined to create one integrated record for each entity.

Traditional integration systems normally solve these tasks using string similarity, manually defined rules, blocking strategies, matching thresholds, clustering algorithms, and voting methods. These techniques are efficient, reproducible, and relatively easy to inspect. However, they may fail when two attributes or product descriptions have the same meaning but share few words.

Large Language Models can interpret the meaning of attribute names and textual descriptions instead of relying only on lexical similarity. They may therefore help identify semantic mappings, analyse difficult product pairs, or choose values when several sources disagree. However, language models also require more computation and memory, and they may produce invalid structured outputs or values that are not supported by the original records.

For this reason, this project follows a hybrid approach. The LLM does not replace the complete traditional pipeline. Deterministic methods are still used for blocking, similarity calculations, clustering, output validation, and fallback decisions. The language model is introduced only in selected stages where semantic reasoning may be useful.

Two complete pipelines are implemented and compared. The traditional baseline uses lexical schema matching, candidate blocking, weighted pairwise similarity, Union-Find clustering, and majority-voting data fusion. The LLM-assisted pipeline follows the same general workflow but

uses Qwen2.5-7B-Instruct for schema alignment, selected borderline record pairs, and a limited number of conflicting clusters.

The experiment uses the WDC Products Multi dataset. Its available Hugging Face version is distributed as a unified table, so three source views are created for this project. Product offers are assigned to `source_alpha`, `source_beta`, and `source_gamma`, and each source uses different attribute names. The original product cluster identifier is preserved as ground truth for record linkage.

The LLM is applied selectively to control memory usage and execution time. Schema alignment requires one call per source. Record-linkage calls are limited to 300 pairs whose similarity scores are close to the matching threshold. Data-fusion calls are limited to 50 conflicting clusters, with no more than three records included in each fusion prompt. The complete experiment takes approximately thirty minutes in Google Colab, although the exact duration depends on the GPU assigned to the session.

The main objective is not to prove that the LLM always produces better results. Instead, the project studies where semantic reasoning is useful, where deterministic methods remain preferable, and how memory requirements, latency, invalid responses, and reproducibility affect the final integration process.

The following sections describe the dataset and source construction, the two integration pipelines, the experimental protocol, the quantitative results, representative errors, and the main conclusions of the comparison.

2 Dataset and Integration Scenario

The project uses the WDC Products Multi dataset, available through Hugging Face. The dataset contains product offers with information such as title, brand, description, price, currency, and a `cluster_id`. This identifier groups records that refer to the same real-world product and is used as the ground truth for record linkage.

The dataset is downloaded automatically with the Hugging Face `datasets` library. Only clusters with at least two records were considered. A maximum of six records was selected from each cluster until the final subset reached 1,800 records and 300 entities.

The final dataset contains:

- 1,800 product records;
- 300 gold entities;
- three source views;
- 600 records per source;
- five mediated attributes;
- 3,600 positive cross-source pairs;
- 1,220 conflicting entity-attribute cases.

The Hugging Face version of the dataset is distributed as one unified table. Since the assignment requires three heterogeneous sources, three artificial source views were created:

- `source_alpha`;
- `source_beta`;
- `source_gamma`.

The records of each entity were distributed among the three sources using a round-robin strategy. Each source contains the same type of product information, but different attribute names are used to create schema heterogeneity.

The mediated schema contains the following attributes:

- `title`;
- `brand`;
- `description`;
- `price`;
- `priceCurrency`.

The attribute names used by each source are the following:

- `source_alpha`: `product_name`, `manufacturer`, `details`, `cost`, and `currency`;
- `source_beta`: `name`, `brand_name`, `description_text`, `amount`, and `currency_code`;
- `source_gamma`: `item_title`, `maker`, `specifications`, `listed_price`, and `money`.

Three reference datasets were created for evaluation. The schema ground truth was obtained directly from the known attribute-renaming rules. The record-linkage ground truth was built from the original `cluster_id`. Two records were considered a positive pair when they belonged to the same entity and came from different source views.

For data fusion, a silver ground truth was generated for each entity. For every attribute, the most frequent normalized value was selected. If several values had the same frequency, the longest value was used as a tie-breaking rule.

The three sources are synthetic views of the same public dataset. Therefore, the experiment includes real differences between product offers, but the source construction is more controlled than in a real integration scenario involving completely independent organizations. This limitation is considered when interpreting the results.

3 Methodology

Two complete integration pipelines were implemented. The first one uses traditional deterministic methods, while the second one adds an LLM to selected stages of the same workflow.

The general process is:

Source records → Schema alignment → Canonicalization → Blocking → Record linkage →
Clustering → Data fusion

3.1 Traditional Baseline

3.1.1 Schema Alignment

The baseline schema alignment compares each source attribute with the attributes of the mediated schema. Attribute names are normalized by converting them to lowercase, removing special characters, and splitting compound names into tokens.

Similarity is computed with RapidFuzz using token-based string comparison. A small set of additional rules is used for common semantic relationships. For example, attributes containing **manufacturer** receive a higher score when compared with **brand**.

The best mapping is accepted when its score is above the selected threshold. Otherwise, the system returns **ABSTAIN**.

3.1.2 Canonicalization

After schema alignment, every source record is converted into the mediated schema. The values are normalized by converting them to lowercase, removing repeated spaces, and replacing missing values with empty strings.

Auxiliary title, brand, and model fields are also extracted from the original attributes. These fields are later used during blocking and record linkage.

3.1.3 Blocking

Comparing every possible pair of records would require more than one million cross-source comparisons. Blocking is therefore used to generate a smaller set of candidate pairs.

The blocking keys are created from:

- normalized brand values;
- product model values;
- relevant title tokens;
- title bigrams;
- combinations of brand and title tokens.

Pairs from the same source are excluded. Blocks containing more than 250 records are also ignored because they are too general and would generate many unrelated comparisons.

3.1.4 Record Linkage

Every candidate pair is compared using title, brand, model, and mediated-attribute similarity.

The final score is calculated as:

$$score = 0.45 \cdot titleSim + 0.20 \cdot brandSim + 0.20 \cdot modelSim + 0.15 \cdot attrSim \quad (1)$$

A pair is classified as a match when its score is equal to or greater than 0.40.

Title and attribute similarity are computed using token-based string similarity. Brand and model fields receive a score of 1 when they are equal and otherwise use the same similarity function.

3.1.5 Clustering

The predicted matching pairs are converted into entity clusters using Union-Find.

When two records are classified as a match, their groups are joined. This also creates transitive matches. For example, if record A matches record B and record B matches record C, the three records are placed in the same cluster.

This method is simple and efficient, although one incorrect match can connect records that should remain in separate entities.

3.1.6 Data Fusion

The baseline creates one integrated record for every predicted cluster.

For each mediated attribute, the most frequent non-empty value is selected. When several values have the same frequency, the longest value is chosen. This rule is used because longer product values often contain more detailed information.

3.2 LLM-Assisted Pipeline

The LLM-assisted pipeline keeps the same blocking, similarity, clustering, and fallback components. The model is only introduced in selected stages.

The model used is:

Qwen/Qwen2.5-7B-Instruct

It is loaded in 4-bit quantized form to reduce GPU memory usage in Google Colab. Generation is deterministic, with sampling disabled and temperature set to zero.

3.2.1 LLM-Assisted Schema Alignment

The LLM receives the mediated schema, the source name, the source attributes, and a small set of example values.

It must return valid JSON containing:

- the original source attribute;
- the selected mediated attribute;
- a confidence value;
- a short reason.

The model can return **ABSTAIN** when no mapping is sufficiently clear.

If the response is invalid or a predicted target does not exist in the mediated schema, the deterministic baseline mapping is used.

3.2.2 LLM-Assisted Record Linkage

The LLM is not used for every candidate pair. It is only applied to borderline cases whose deterministic score is between 0.35 and 0.50.

A maximum of 300 pairs is sent to the model. Each prompt contains the main title, brand, model, and mediated attributes of the two records.

The model returns JSON with:

- a match decision;
- a confidence value;
- a short explanation.

When the response is valid, the LLM decision replaces the original deterministic decision. If the response is invalid, the original score decision is preserved.

3.2.3 LLM-Assisted Data Fusion

The LLM is also applied to a limited number of predicted clusters that contain conflicting values.

At most 50 clusters are selected, and each prompt contains no more than three records. The model must return one value for each mediated attribute.

The prompt explicitly asks the model to copy values exactly from the provided records and not to shorten, combine, translate, infer, or invent information.

Every proposed value is checked against the values found in the cluster. If a value is not supported, it is rejected and the majority-voting value is kept instead.

3.2.4 Structured Outputs and Fallbacks

All LLM stages use JSON outputs. The raw response, parsed response, success status, prompt, and metadata are stored in a log file.

The fallback policy is:

- baseline schema mapping for invalid schema responses;
- deterministic match decision for invalid linkage responses;
- majority voting for invalid or unsupported fusion outputs.

No LLM output is manually corrected before evaluation.

4 Experimental Protocol

The complete experiment was executed in Google Colab using a GPU runtime. The exact GPU model may change between sessions because it is assigned automatically by Colab. The total execution time was approximately thirty minutes.

The language model was loaded using 4-bit quantization. This reduced GPU memory consumption and made it possible to execute Qwen2.5-7B-Instruct in the Colab environment.

During development, larger prompts and a higher number of fusion calls caused memory pressure. For this reason, the final implementation uses compact prompts, a maximum of three records per fusion prompt, and a limited number of LLM calls.

The random seed was fixed to 42 for Python and NumPy. The model was executed without sampling and with a temperature of zero. This makes the generation more stable, although small differences may still appear depending on the GPU, CUDA version, and library versions.

The main experimental parameters are shown in Table 1.

Table 1: Main experimental parameters

Parameter	Value
Random seed	42
Matching threshold	0.40
Borderline lower limit	0.35
Borderline upper limit	0.50
Maximum block size	250
Maximum schema attributes per prompt	35
Maximum linkage calls	300
Maximum fusion calls	50
Maximum records per fusion prompt	3
Schema maximum output tokens	768
Linkage maximum output tokens	240
Fusion maximum output tokens	384
Temperature	0.0
Sampling	Disabled

The experiment was executed once from a clean runtime. This was important because the LLM log is stored in memory during execution. Re-executing only the LLM cells without resetting the runtime could duplicate the logged calls.

The final execution produced 353 LLM calls:

- 3 calls for schema alignment;
- 300 calls for record linkage;
- 50 calls for data fusion.

Schema alignment was evaluated using accuracy, precision, recall, and F1 score.

Record linkage was evaluated at pair level using precision, recall, and F1 score. The number of candidate pairs and the blocking reduction ratio were also measured.

The reduction ratio was calculated by comparing the number of candidate pairs with the total number of possible cross-source pairs.

Data fusion was evaluated by comparing the selected values with the silver ground truth. The metric reports the proportion of correctly selected attribute values.

The baseline and LLM-assisted pipelines may produce different clusters. Therefore, the number of evaluated fusion values is not necessarily the same for both pipelines. The fusion metric must be interpreted as an end-to-end integrated-value measure because it is also affected by schema alignment, record linkage, and clustering.

All LLM prompts and responses were stored in a JSONL log file. Invalid JSON responses, missing structures, and unsupported fusion values were treated as failures. The system then used the deterministic fallback defined for the corresponding stage.

No model response was manually corrected before calculating the metrics.

5 Results

5.1 Schema Alignment

The traditional schema-alignment method correctly mapped 6 of the 15 source attributes. The remaining nine attributes were classified as **ABSTAIN**. It obtained the following results:

- Accuracy: 0.400
- Precision: 1.000
- Recall: 0.400
- F1 score: 0.571

The baseline did not produce incorrect mappings, but it failed to map several attributes whose names were semantically related but lexically different.

The LLM correctly mapped all 15 source attributes and obtained:

- Accuracy: 1.000
- Precision: 1.000
- Recall: 1.000
- F1 score: 1.000

The largest improvement appeared in mappings such as **manufacturer** to **brand**, **details** to **description**, and **listed_price** to **price**. These cases show that the LLM was able to interpret the semantic meaning of the source attributes.

5.2 Blocking

The complete dataset contains 1,080,000 possible cross-source record pairs. The blocking stage reduced this number to 33,855 candidate pairs.

This represents a reduction ratio of 96.87%. Therefore, only 3.13% of the possible cross-source pairs had to be evaluated during record linkage.

The candidate set retained 3,484 of the 3,600 gold positive pairs. Blocking therefore preserved approximately 96.78% of the known matches while removing most unrelated comparisons.

5.3 Record Linkage

The traditional baseline produced the following results:

- Predicted matches: 7,738
- True positives: 2,357
- False positives: 5,381
- False negatives: 1,243
- Precision: 0.305
- Recall: 0.655
- F1 score: 0.416

The LLM-assisted pipeline produced:

- Predicted matches: 6,547
- True positives: 2,079
- False positives: 4,468
- False negatives: 1,521
- Precision: 0.318
- Recall: 0.578
- F1 score: 0.410

The LLM-assisted pipeline predicted fewer matches. This reduced the number of false positives and increased precision from 0.305 to 0.318. However, it also rejected several true matches, causing recall to decrease from 0.655 to 0.578.

As a result, the final F1 score decreased slightly from 0.416 to 0.410. The LLM therefore behaved as a conservative validator: it removed some incorrect matches, but also rejected some correct ones.

The baseline generated 140 predicted clusters, while the LLM-assisted pipeline generated 197. The gold reference contains 300 entities.

Both pipelines therefore produced fewer clusters than expected. This indicates that some separate entities were incorrectly joined through transitive Union-Find connections. The LLM-assisted pipeline was closer to the correct number because it predicted fewer pairwise matches.

5.4 Data Fusion

The traditional pipeline correctly selected 136 of 679 evaluated attribute values. Its integrated-value accuracy was 0.200.

The final LLM-assisted pipeline correctly selected 229 of 958 evaluated values. Its integrated-value accuracy was 0.239.

The number of evaluated values is different because the two pipelines produce different schema mappings and different predicted clusters. For this reason, the fusion metric also reflects the effects of schema alignment, record linkage, and clustering.

Before applying generative LLM fusion, majority voting over the LLM-assisted clusters correctly selected 273 of the same 958 values. This corresponds to an accuracy of 0.285.

After the selective LLM fusion calls, the accuracy decreased to 0.239. Therefore, the complete LLM-assisted pipeline performs better than the traditional baseline under the reported end-to-end metric, but the generative fusion stage did not improve the majority-voting result obtained on the same clusters.

5.5 LLM Calls and Output Validation

The final execution produced 353 LLM calls:

- 3 calls for schema alignment
- 300 calls for record linkage
- 50 calls for data fusion

All schema-alignment and record-linkage calls returned valid structured responses.

Among the 50 data-fusion calls, 32 responses were fully valid. Two responses contained invalid JSON, and 16 responses included at least one unsupported value.

A total of 22 unsupported attribute values were detected. These values were rejected because they did not appear in the corresponding source records. In these cases, the system preserved the deterministic majority-voting result.

The validation and fallback strategy prevented invalid JSON and unsupported LLM values from being inserted directly into the final integrated dataset.

6 Error Analysis

The generated result files were inspected to identify representative errors. The examples below show that integration failures do not come from a single stage. Some errors are caused directly by the LLM, while others are inherited from blocking, pairwise matching, or clustering.

6.1 Example 1: False Positive in Record Linkage

The first record was:

Record A: Tissot T-Sport Seastar 1000 Powermatic men’s watch, model T1204071704100.

The second record was:

Record B: Tissot T-Sport Seastar 1000 Chronograph black dial watch, model T120.417.11.051.00.

The deterministic similarity score was 0.492. Since this value was inside the borderline interval, the pair was sent to the LLM.

The model returned a match decision with confidence 0.8. Its explanation stated that both records referred to the Tissot T-Sport Seastar 1000 chronograph series.

The expected decision was a non-match. Although both watches belong to the same brand and product family, they have different model numbers and technical characteristics. The first record describes an automatic Powermatic model, while the second describes a quartz chronograph.

The LLM focused too strongly on the shared brand and product family and did not give enough importance to the different model numbers. This example shows that semantic similarity can produce false positives when two products belong to the same series.

6.2 Example 2: False Negative Caused by Language Variation

The two source titles were:

Record A: Kappe New Era Clean Trucker Chicago Bulls cap.

Record B: Sapca New Era Clean Trucker Chicago Bulls cap.

Both records belong to the same gold entity. Their deterministic similarity score was 0.494, so the pair was evaluated by the LLM.

The model returned a non-match decision with confidence 0.0. Its reason was that the titles were different because one used “kappe” and the other used “sapca”.

The expected decision was a match. The two different words refer to the same type of product in different languages, while the remaining brand, team, model, and product information is identical.

This error shows that the LLM did not correctly interpret the multilingual variation in this case. It treated a translation difference as evidence of a different product, even though the rest of the title strongly supported a match.

6.3 Example 3: Fusion Error after Incorrect Clustering

A predicted cluster contained these records:

Record A: Western Digital My Passport 1TB, model WDBYVG0010BBK-WESN, price 317.99 PLN.

Record B: Western Digital My Passport 1TB, model WDBYNN0010BBK-WESN, price 46383 CLP.

The records describe similar products, but their model identifiers and gold entity identifiers are different. They were incorrectly joined before the fusion stage.

The LLM returned the shortened title “My Passport 1TB 2.5 black WDBYVG0010BBK-WESN”, inferred the brand “Western Digital”, and selected the price 46383 with currency CLP.

For the entity selected during evaluation, the expected price was 317.99 and the expected currency was PLN.

The generated title and brand were not exact values from the cluster, so they were rejected by the validation step. However, the price and currency were present in Record B and were therefore considered supported, even though Record B belonged to a different gold entity.

This example demonstrates that data fusion cannot completely correct an incorrect cluster. The validation method can prevent invented values, but it cannot determine whether a supported value comes from a record that should not have been included in the cluster.

6.4 General Error Patterns

The record-linkage errors show two opposite behaviours. In some cases, the LLM accepted products from the same family even when their models were different. In other cases, it rejected true matches because of language variation, formatting differences, or different prices.

The fusion logs also showed reliability problems. Two of the 50 fusion responses contained invalid JSON. Sixteen additional responses contained at least one unsupported value, and a total of 22 proposed values were rejected.

These cases confirm the importance of storing the raw outputs, validating every generated value, and using deterministic fallback rules. Without these checks, invalid or unsupported information could have been inserted directly into the integrated dataset.

7 Discussion

The results show that the usefulness of the LLM depends strongly on the integration stage.

The clearest improvement appeared in schema alignment. The traditional method obtained an F1 score of 0.571 because it depended mainly on lexical similarity and abstained when attribute names were too different. The LLM correctly mapped all source attributes because it was able to interpret relationships such as `manufacturer` and `brand`, or `specifications` and `description`.

Record linkage produced a less positive result. The LLM-assisted pipeline slightly improved precision, but recall decreased. This means that the model removed some false matches, but also rejected several correct pairs. The final F1 score was therefore slightly lower than the baseline score.

This behaviour suggests that the LLM acted as a conservative validator. It was useful when the deterministic system accepted a pair based on superficial similarity, but it was less reliable when the records contained multilingual descriptions, formatting differences, incomplete information, or large price differences.

The results also show the importance of model identifiers. Products from the same brand and family may have very similar titles while representing different models. In these cases, semantic similarity alone is not sufficient. Exact identifiers and structured attributes must receive high importance in the final decision.

The data-fusion stage produced another important observation. The complete LLM-assisted pipeline obtained a higher integrated-value accuracy than the traditional pipeline. However, majority voting over the same LLM-assisted clusters obtained a higher score than the final generative fusion result.

Therefore, the improvement of the complete LLM-assisted pipeline came mainly from better schema alignment and different clustering, rather than from generative fusion itself.

Majority voting performed well because the fusion silver truth was also generated using the most frequent value. This gives the deterministic strategy an advantage. At the same time, the LLM fusion sometimes shortened values, normalized product names, inferred brands, or selected information from records that had been incorrectly included in a cluster.

The validation step was essential. Unsupported values were rejected and replaced with the majority-voting result. Without this control, the integrated dataset would have contained generated values that were not present in the original records.

However, validation cannot solve every problem. If an incorrect record is already inside a cluster, its values are considered supported. The fusion system may therefore select a valid value from the wrong entity. This confirms that errors produced during record linkage and clustering directly affect the quality of data fusion.

The LLM-assisted pipeline also required more computational resources than the baseline. The complete experiment used 353 model calls and required approximately thirty minutes in Google Colab. The exact time depends on the GPU assigned to the session.

Memory consumption was another practical limitation. The model had to be loaded using 4-bit quantization. The number of linkage and fusion calls was limited, prompts were shortened, and no more than three records were included in each fusion prompt.

The deterministic pipeline is faster and easier to reproduce because its results depend only on fixed rules and thresholds. The LLM was executed with temperature zero and sampling disabled, but exact reproducibility may still depend on the model version, hardware, CUDA environment, and library versions.

The main limitations of the experiment are:

- The three sources are synthetic views of one unified dataset rather than independent real-world providers.
- Schema heterogeneity was mainly introduced by renaming attributes.
- The fusion reference was generated automatically and favours majority voting.

- Only 300 borderline pairs were evaluated by the LLM.
- Only 50 conflicting clusters received generative fusion.
- Matching thresholds were selected experimentally and were not tuned on a separate validation set.
- The model was executed in 4-bit quantized form because of Colab memory limitations.
- The experiment used one final clean execution rather than several repeated runs.

Overall, the experiment does not show that an LLM should replace traditional integration methods. It shows that the strongest design is hybrid.

The LLM is particularly useful for semantic interpretation, while deterministic methods remain important for blocking, exact identifier comparison, clustering control, output validation, and fallback decisions.

8 Discussion

The results show that the usefulness of the LLM depends strongly on the integration stage.

The clearest improvement appeared in schema alignment. The traditional method obtained an F1 score of 0.571 because it depended mainly on lexical similarity and abstained when attribute names were too different. The LLM correctly mapped all source attributes because it was able to interpret relationships such as `manufacturer` and `brand`, or `specifications` and `description`.

Record linkage produced a less positive result. The LLM-assisted pipeline slightly improved precision, but recall decreased. This means that the model removed some false matches, but also rejected several correct pairs. The final F1 score was therefore slightly lower than the baseline score.

This behaviour suggests that the LLM acted as a conservative validator. It was useful when the deterministic system accepted a pair based on superficial similarity, but it was less reliable when the records contained multilingual descriptions, formatting differences, incomplete information, or large price differences.

The results also show the importance of model identifiers. Products from the same brand and family may have very similar titles while representing different models. In these cases, semantic similarity alone is not sufficient. Exact identifiers and structured attributes must receive high importance in the final decision.

The data-fusion stage produced another important observation. The complete LLM-assisted pipeline obtained a higher integrated-value accuracy than the traditional pipeline. However, majority voting over the same LLM-assisted clusters obtained a higher score than the final generative fusion result.

Therefore, the improvement of the complete LLM-assisted pipeline came mainly from better schema alignment and different clustering, rather than from generative fusion itself.

Majority voting performed well because the fusion silver truth was also generated using the most frequent value. This gives the deterministic strategy an advantage. At the same time, the LLM fusion sometimes shortened values, normalized product names, inferred brands, or selected information from records that had been incorrectly included in a cluster.

The validation step was essential. Unsupported values were rejected and replaced with the majority-voting result. Without this control, the integrated dataset would have contained generated values that were not present in the original records.

However, validation cannot solve every problem. If an incorrect record is already inside a cluster, its values are considered supported. The fusion system may therefore select a valid value from the wrong entity. This confirms that errors produced during record linkage and clustering directly affect the quality of data fusion.

The LLM-assisted pipeline also required more computational resources than the baseline. The complete experiment used 353 model calls and required approximately thirty minutes in Google Colab. The exact time depends on the GPU assigned to the session.

Memory consumption was another practical limitation. The model had to be loaded using 4-bit quantization. The number of linkage and fusion calls was limited, prompts were shortened, and no more than three records were included in each fusion prompt.

The deterministic pipeline is faster and easier to reproduce because its results depend only on fixed rules and thresholds. The LLM was executed with temperature zero and sampling disabled, but exact reproducibility may still depend on the model version, hardware, CUDA environment, and library versions.

The main limitations of the experiment are:

- The three sources are synthetic views of one unified dataset rather than independent real-world providers.
- Schema heterogeneity was mainly introduced by renaming attributes.
- The fusion reference was generated automatically and favours majority voting.
- Only 300 borderline pairs were evaluated by the LLM.
- Only 50 conflicting clusters received generative fusion.
- Matching thresholds were selected experimentally and were not tuned on a separate validation set.
- The model was executed in 4-bit quantized form because of Colab memory limitations.
- The experiment used one final clean execution rather than several repeated runs.

Overall, the experiment does not show that an LLM should replace traditional integration methods. It shows that the strongest design is hybrid.

The LLM is particularly useful for semantic interpretation, while deterministic methods remain important for blocking, exact identifier comparison, clustering control, output validation, and fallback decisions.

9 Reproducibility and Repository

The complete implementation is available in the following GitHub repository:

<https://github.com/JuanMiguelPinos/BigDataIntegration.git>

The repository contains the source code, execution notebook, generated source views, ground-truth files, prompts, model configuration, evaluation metrics, LLM logs, and final integrated datasets.

The main project contents are organized as follows:

- **data/**: source views, processed records, and ground-truth files;
- **prompts/**: prompt templates used for schema alignment, record linkage, and data fusion;
- **results/**: evaluation metrics, predicted clusters, integrated datasets, and LLM logs;
- **notebooks/**: notebook used to execute the complete experiment;
- **requirements.txt**: required Python libraries;

- `README.md`: installation and execution instructions.

The dataset is downloaded automatically from Hugging Face. The experiment can be reproduced by installing the required libraries and executing the notebook from beginning to end in a Google Colab GPU runtime.

The random seed is fixed to 42, and LLM generation uses temperature zero with sampling disabled. A clean runtime should be used to avoid duplicating model calls or previously generated log entries.

The exact execution time and memory consumption may vary depending on the GPU assigned by Google Colab. In the final experiment, the complete pipeline required approximately thirty minutes.